

EXP 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory

1. What is Blockchain?

A blockchain is a shared, decentralized ledger made up of a chain of blocks. Every block holds:

- A set of transactions
- A timestamp of creation
- The hash of the block before it
- Its own hash (a unique fingerprint of that block's data)

Because each block's hash depends on the previous one, the chain becomes tamper-resistant — changing any single block would force you to redo the hashes of every block after it.

2. Mining Explained

Mining is the process by which new blocks get added to the chain:

1. Pending transactions waiting in the pool are gathered.
2. A computational puzzle (Proof-of-Work) is solved to generate a valid hash.
3. The newly formed block is appended to the chain.
4. The update is shared with all other nodes in the network.

As an incentive, whoever mines the block receives a small reward in the form of coins.

3. Running Multiple Nodes

To mimic a real decentralized network, three separate nodes are set up on ports 5001, 5002, and 5003.

Each one:

- Operates independently on its own port
- Keeps its own local copy of the chain
- Can link up with other nodes to exchange and check blocks

4. Reaching Agreement (Consensus)

The network follows the **Longest Chain Rule** — whenever nodes disagree on which version of the chain is correct, the longest valid one wins, keeping everyone's records in sync.

5. Handling Transactions

Every transaction record contains:

- Who sent it
- Who received it
- The amount transferred

When a block is mined, any transactions waiting in the pool get bundled into it, plus one extra transaction that pays the miner their reward.

6. Keeping Chains in Sync

Calling `/replace_chain` triggers this process:

1. The node asks its peers for their current chains.
2. If a peer has a longer, valid chain, the node adopts it.
3. This keeps every node's copy of the ledger consistent.

Code:

```
import hashlib, json, datetime
from uuid import uuid4
from urllib.parse import urlparse
from flask import Flask, jsonify, request
import requests

class Blockchain:
    def __init__(self):
        self.chain = []
        self.pending_tx = []
        self.nodes = set()
        self.add_block(proof=1, prev_hash='0')

    def add_block(self, proof, prev_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp':
str(datetime.datetime.now()),
            'proof': proof,
            'prev_hash': prev_hash,
            'transactions': self.pending_tx
        }
        self.pending_tx = []
        self.chain.append(block)
        return block

    def last_block(self):
        return self.chain[-1]

    def hash_block(self, block):
        return hashlib.sha256(json.dumps(block,
sort_keys=True).encode()).hexdigest()

    def mine(self, prev_proof):
        proof = 1
        while hashlib.sha256(str(proof**2 -
prev_proof**2).encode()).hexdigest()[:4] !=
'0000':
            proof += 1
        return proof

    def is_valid(self, chain):
        for i in range(1, len(chain)):
            prev, curr = chain[i - 1], chain[i]
            if curr['prev_hash'] !=
self.hash_block(prev):
                return False
            check =
hashlib.sha256(str(curr['proof']**2 -
prev['proof']**2).encode()).hexdigest()
            if check[:4] != '0000':
                return False
        return True

    def add_transaction(self, sender, receiver,
amount):
        self.pending_tx.append({'sender': sender,
'receiver': receiver, 'amount': amount})
        return self.last_block()['index'] + 1

    def add_node(self, address):
        self.nodes.add(urlparse(address).netloc)

    def sync_chain(self):
```

```

        longest, max_len = None, len(self.chain)
        for node in self.nodes:
            r =
requests.get(f'http://{node}/get_chain')
            if r.status_code == 200:
                data = r.json()
                if data['length'] > max_len and
self.is_valid(data['chain']):
                    max_len, longest = data['length'],
data['chain']
                if longest:
                    self.chain = longest
                    return True
                return False

```

```

app = Flask(__name__)
node_id = str(uuid4()).replace('-', '')
bc = Blockchain()

```

```

@app.route('/mine_block', methods=['GET'])
def mine_block():
    prev = bc.last_block()
    proof = bc.mine(prev['proof'])
    bc.add_transaction(node_id, 'Richard', 1) #
mining reward
    block = bc.add_block(proof,
bc.hash_block(prev))
    return jsonify(block), 200

```

```

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({'chain': bc.chain, 'length':
len(bc.chain)}), 200

```

```

@app.route('/is_valid', methods=['GET'])
def check_valid():

```

```

    ok = bc.is_valid(bc.chain)
    return jsonify({'message': 'Valid chain' if ok
else 'Invalid chain'}), 200

```

```

@app.route('/add_transaction',
methods=['POST'])
def new_transaction():
    data = request.get_json()
    if not all(k in data for k in ('sender', 'receiver',
'amount')):
        return 'Missing fields', 400
    index = bc.add_transaction(data['sender'],
data['receiver'], data['amount'])
    return jsonify({'message': f'Transaction will
be added to block {index}'}), 201

```

```

@app.route('/connect_node', methods=['POST'])
def connect_node():
    nodes = request.get_json().get('nodes')
    if not nodes:
        return 'No nodes provided', 400
    for n in nodes:
        bc.add_node(n)
    return jsonify({'message': 'Nodes connected',
'total_nodes': list(bc.nodes)}), 201

```

```

@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    replaced = bc.sync_chain()
    msg = 'Chain replaced with a longer valid one'
    if replaced else 'Current chain is already the
longest'
    return jsonify({'message': msg, 'chain':
bc.chain}), 200

```

```

app.run(host='0.0.0.0', port=5001)

```

Output:

HTTP **http://127.0.0.1:5001/connect_node**

POST **http://127.0.0.1:5001/connect_node**

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5002",
4     "http://127.0.0.1:5003"
5   ]
6 }
7
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON ▾ ▶ Preview Visualize ▾

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5003",
5     "127.0.0.1:5002"
6   ]
7 }
```

HTTP **http://127.0.0.1:5002/connect_node**

POST **http://127.0.0.1:5002/connect_node**

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
7
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON ▾ ▶ Preview Visualize ▾

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5003",
5     "127.0.0.1:5001"
6   ]
7 }
8
```

HTTP **http://127.0.0.1:5001/add_transaction**

POST **http://127.0.0.1:5001/add_transaction**

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "sender": "Nami",
3   "receiver": "Zoro",
4   "amount": 10
5 }
6
7
```

Body Cookies Headers 5 Test Results | ↺

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
4
```

HTTP **http://127.0.0.1:5001/add_transaction**

POST **http://127.0.0.1:5001/add_transaction**

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "sender": "Bob",
3   "receiver": "Charlie",
4   "amount": 5
5 }
6
```

Body Cookies Headers 5 Test Results | ↺

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
4
```

 http://127.0.0.1:5001/mine_block

GET http://127.0.0.1:5001/mine_block

Docs Params Authorization Headers 8 Body Scripts Settings

Body Cookies Headers 5 Test Results

{}

JSON

Preview

Visualize

```
1  {
2    "index": 2,
3    "message": "Congratulations, you just mined a block!",
4    "previous_hash": "0bea07bc23dfa453ad0f276448099241baff5c81c27340c439fe542ddf4c3a82",
5    "proof": 533,
6    "timestamp": "2026-09-02 12:43:10.825990",
7    "transactions": [
8      {
9        "amount": 10,
10       "receiver": "Zoro",
11       "sender": "Nami"
12     },
13     {
14       "amount": 5,
15       "receiver": "Charlie",
16       "sender": "Bob"
17     },
18     {
19       "amount": 1,
20       "receiver": "Richard",
21       "sender": "0dbb60fbe08942859ef1e6ba112f94e0"
22     }
23   ]
}
```



http://127.0.0.1:5001/get_chain

GET

http://127.0.0.1:5001/get_chain

Docs Params Authorization Headers 8 Body Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:28:04.400389",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "0bea07bc23dfa453ad0f276448099241baff5c81c27340c439fe542ddf4c3a82",
13      "proof": 533,
14      "timestamp": "2026-09-02 12:43:10.825990",
15      "transactions": [
16        {
17          "amount": 10,
18          "receiver": "Zoro",
19          "sender": "Nami"
20        },
21        {
22          "amount": 5,
23          "receiver": "Charlie",
24          "sender": "Bob"
25        },
26        {
27          "amount": 1,
28          "receiver": "Richard",
29          "sender": "0d6bb60f6e08942859ef1e6ba112f94e0"
30        }
31      ]
32    }
33  ],
34  "length": 2
35 }
36
```

HTTP http://127.0.0.1:5002/get_chain

GET http://127.0.0.1:5002/get_chain

Docs Params Authorization Headers 8 Body • Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:30:47.165549",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
13
```

HTTP http://127.0.0.1:5003/get_chain

GET http://127.0.0.1:5003/get_chain

Docs Params Authorization Headers 8 Body • Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:31:28.739765",
8       "transactions": []
9     }
10  ],
11  "length": 1
12 }
13
```

5001 → 2 blocks

5002 → 1 block

5003 → 1 block



http://127.0.0.1:5002/replace_chain

GET

http://127.0.0.1:5002/replace_chain

Docs Params Authorization Headers 5 Body Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-09-02 12:28:04.400389",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash": "0bea07bc23dfa453ad0f276448099241baff5c81c27340c439fe542ddf4c3a82",
14      "proof": 533,
15      "timestamp": "2026-09-02 12:43:10.825990",
16      "transactions": [
17        {
18          "amount": 10,
19          "receiver": "Zoro",
20          "sender": "Nami"
21        },
22        {
23          "amount": 5,
24          "receiver": "Charlie",
25          "sender": "Bob"
26        },
27        {
28          "amount": 1,
29          "receiver": "Richard",
30          "sender": "0ddb60fbc08942859ef1e6ba112f94e0"
31        }
32      ]
33    }
34  ]
35 }
```

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-09-02 12:28:04.406389",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "0bea07bc23dfa453ad0f276448099241baff5c81c27340c439fe542ddf4c3a82",
13      "proof": 533,
14      "timestamp": "2026-09-02 12:43:10.825990",
15      "transactions": [
16        {
17          "amount": 10,
18          "receiver": "Zoro",
19          "sender": "Nami"
20        },
21        {
22          "amount": 5,
23          "receiver": "Charlie",
24          "sender": "Bob"
25        },
26        {
27          "amount": 1,
28          "receiver": "Richard",
29          "sender": "0d5b60fbc08942859ef1e6ba112f94e0"
```

```
1 {
2   "message": "All good. The Blockchain is valid."
3 }
```

Conclusion

In this experiment, a basic cryptocurrency was built using Python, and mining was carried out across a network of three simulated nodes. Each node kept an independent copy of the ledger and stayed synchronized with the others through the Longest Chain Rule. Blocks were mined using Proof-of-Work, which allowed transactions to be added securely while rewarding the miner for their effort. Overall, this exercise illustrated core blockchain principles — decentralized consensus, validation of transactions, and keeping multiple nodes in sync.